

Лекция 1. Введение

1 О чём предмет?

Возьмём простой код:

```
#include <iostream>

int main() {
    int x;
    std::cin >> x;
    std::cout << x + 5;
    return 0;
}
```

Скомпилируем его командой `g++ main.cpp`.

Получаем честный исполняемый файл в отличие от Java. Можем написать `cat a.out` и получить белиберду (не совсем), а можем `hexdump -c a.out` и получить бинарный формат (флагом включается отображение соответствующего байтам текста справа).

Вот мы вводим 5, выводим 10. Наша задача — понять, как и откуда это получилось. Вариант “взяли из `iostream`” нас не устраивает: это мы должны знать с первого курса, сейчас нас интересует то, как наш ввод стал значениями переменных и как вычисленное значение попало в консоль.

2 Этапы сборки

2.1 Препроцессинг

Компиляция происходит в несколько стадий. Первая стадия — препроцессинг. Пишем `g++ -E [-v] main.cpp > main.e` и получаем результат стадии препроцессинга. Флаг `-v` добавляет подробный вывод: по нему можно посмотреть, откуда берутся библиотеки.

```
#include "... " search starts here:
#include <...> search starts here:
/usr/local/include
.
/usr/include/c++/14
/usr/include/x86_64-linux-gnu/c++/14
/usr/include/c++/14/backward
/usr/lib/gcc/x86_64-linux-gnu/14/include
/usr/include/x86_64-linux-gnu
/usr/include
```

Идём в `iostream` и видим:

```
extern istream cin;          ///< Linked to standard input
extern ostream cout;        ///< Linked to standard output
extern ostream cerr;        ///< Linked to standard error (unbuffered)
extern ostream clog;        ///< Linked to standard error (buffered)
```

Где-то на 60-х строчках лежат объявления переменных. Слово `extern` говорит нам о том, что эти переменные обязательно будут, но где-то там, далеко-далеко, а сейчас у нас есть только имя и тип (подробнее [здесь](#)). Потом греем по `main.e` и видим, что определения не появились.

Когда мы пишем `include`, мы подключаем только заголовочные файлы. При сборке потом происходит линковка, в ходе которой подключаются уже собранные реализации. На чистой системе исходного кода этих функций нет.

2.2 Компиляция

Теперь посмотрим следующую стадию — компиляцию:

```
g++ -S main.cpp
```

Команда `wc -l <file>` позволяет посчитать количество строк в файле. Заметим, что здесь уже сильно меньше строк: порядка 120, тогда как в `main.e` больше 30 тысяч.

Среди других инструкций найдём `call`. Это вызовы наших функций; в таком виде они записываются в ассемблере. Здесь мы видим странные имена: они изменены из-за того, что в C++ есть перегрузка функций, классы, операторы и неймспейсы. Чтобы избежать дублирования имён, компилятор изменяет имена таким образом. Это называется *name mangling* или *name decoration* (подробнее [здесь](#)).

Команда `c++filt <mangled name>` возвращает исходное имя объекта.

2.3 Ассемблирование

После этой стадии идёт ассемблирование, которое переводит файл из `.s` в `.o`:

```
g++ -c main.cpp
```

В нём инструкции просто отображаются в бинарный формат. Мы можем легко отобразить объектный файл обратно в ассемблерный:

```
objdump -d main.o
```

Но `main.o` мы всё ещё не можем исполнить. Почему? Потому что в нём нет реализаций внешних функций.

2.4 Линковка

Соберём `main.cpp` полностью командой `g++ main.cpp` и получим файл `a.out`. Напишем `objdump -d a.out`, найдём `main` и наконец-то увидим рядом с `call` адрес.

Попросим подробный вывод (ключ `-v`) и увидим команду компиляции:

```
/usr/libexec/gcc/x86_64-linux-gnu/14/cc1plus -quiet -v -imultarch x86_64-  
linux-gnu -D_GNU_SOURCE main.cpp -quiet -dumpdir a- -dumpbase main.cpp -  
dumpbase-ext .cpp -mtune=generic -march=x86-64 -version -foffload-  
options=-l_GCC_m -fasynchronous-unwind-tables -fstack-protector-strong -  
Wformat -Wformat-security -fstack-clash-protection -fcf-protection -o /  
tmp/cch9iE9S.s
```

Затем команду ассемблирования:

```
as -v --64 -o /tmp/ccpWsqa.o /tmp/cch9iE9S.s
```

И, наконец, команду линковки:

```
/usr/libexec/gcc/x86_64-linux-gnu/14/collect2 -plugin /usr/libexec/gcc/  
x86_64-linux-gnu/14/liblto_plugin.so -plugin-opt=/usr/libexec/gcc/x86_64-  
linux-gnu/14/lto-wrapper -plugin-opt=-fresolution=/tmp/ccvUpmp.res -  
plugin-opt=-pass-through=-lgcc_s -plugin-opt=-pass-through=-lgcc -plugin-  
opt=-pass-through=-lc -plugin-opt=-pass-through=-lgcc_s -plugin-opt=-  
pass-through=-lgcc --build-id --eh-frame-hdr -m elf_x86_64 --hash-style=  
gnu --as-needed -dynamic-linker /lib64/ld-linux-x86-64.so.2 -pie -z now  
-z relro /usr/lib/gcc/x86_64-linux-gnu/14/../../../../x86_64-linux-gnu/  
Scrt1.o /usr/lib/gcc/x86_64-linux-gnu/14/../../../../x86_64-linux-gnu/crti.  
o /usr/lib/gcc/x86_64-linux-gnu/14/crtbeginS.o -L/usr/lib/gcc/x86_64-  
linux-gnu/14 -L/usr/lib/gcc/x86_64-linux-gnu/14/../../../../x86_64-linux-  
gnu -L/usr/lib/gcc/x86_64-linux-gnu/14/../../../../lib -L/lib/x86_64-  
linux-gnu -L/lib/./lib -L/usr/lib/x86_64-linux-gnu -L/usr/lib/./lib -L  
/usr/lib/gcc/x86_64-linux-gnu/14/../../../../tmp/ccpWsqa.o -lstdc++ -lm -  
lgcc_s -lgcc -lc -lgcc_s -lgcc /usr/lib/gcc/x86_64-linux-gnu/14/crtendS.  
o /usr/lib/gcc/x86_64-linux-gnu/14/../../../../x86_64-linux-gnu/crtn.o
```

3 Исполняемые файлы и библиотеки

3.1 Динамические библиотеки

Есть команда `ldd`, которая позволяет посмотреть динамические библиотеки, необходимые для запуска бинаря:

```
linux-vdso.so.1  
libstdc++.so.6 => /lib/x86_64-linux-gnu/libstdc++.so.6  
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6  
libm.so.6 => /lib/x86_64-linux-gnu/libm.so.6  
/lib64/ld-linux-x86-64.so.2  
libgcc_s.so.1 => /lib/x86_64-linux-gnu/libgcc_s.so.1
```

Воспользуемся командой `file`:

```
/lib/x86_64-linux-gnu/libstdc++.so.6: symbolic link to libstdc++.so.6.0.34
/lib/x86_64-linux-gnu/libstdc++.so.6.0.34: ELF 64-bit LSB shared object,
      x86-64, version 1 (GNU/Linux), dynamically linked, BuildID[sha1]=8
      d4f2235ec34ae33c412aa436c18ef4618f2efa6, stripped
a.out: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
      linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=
      ba1ae9a1317b2e65de6eab8d0c4c45f0c952dd77, for GNU/Linux 3.2.0, not
      stripped
```

Посмотрим, что они все формата ELF (*Executable and Linkable Format*). Это формат бинарных файлов во всех Unix-системах.

3.2 ELF

Есть тулза `readelf`, она позволяет посмотреть информацию о таких файлах. Например:

```
readelf -h a.out
```

Заголовок исполняемого файла:

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                      2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                UNIX - System V
  ABI Version:                           0
  Type:                                  DYN (Position-Independent Executable
      file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x10a0
  Start of program headers:              64 (bytes into file)
  Start of section headers:              14472 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              14
  Size of section headers:               64 (bytes)
  Number of section headers:              32
  Section header string table index:     31
```

Заголовок динамической библиотеки:

ELF Header:

```

Magic:   7f 45 4c 46 02 01 01 03 00 00 00 00 00 00 00
Class:                                     ELF64
Data:                                     2's complement, little endian
Version:                                1 (current)
OS/ABI:                                 UNIX - GNU
ABI Version:                             0
Type:                                    DYN (Shared object file)
Machine:                                Advanced Micro Devices X86-64
Version:                                0x1
Entry point address:                     0x0
Start of program headers:                64 (bytes into file)
Start of section headers:               2651456 (bytes into file)
Flags:                                   0x0
Size of this header:                     64 (bytes)
Size of program headers:                 56 (bytes)
Number of program headers:               13
Size of section headers:                 64 (bytes)
Number of section headers:               35
Section header string table index:      34

```

Те объекты, которыми оперирует линковщик, называются символами. Смотрим символы файла командой `readelf -s`. Например, `main.o`:

Symbol table '.symtab' contains 13 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	main.cpp
2:	0000000000000000	0	SECTION	LOCAL	DEFAULT	1	.text
3:	0000000000000000	1	OBJECT	LOCAL	DEFAULT	5	_ZNSt8__detail30
[...]							
4:	0000000000000001	1	OBJECT	LOCAL	DEFAULT	5	_ZNSt8__detail30
[...]							
5:	0000000000000002	1	OBJECT	LOCAL	DEFAULT	5	_ZNSt8__detail30
[...]							
6:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	_ZSt21ios_base_1
[...]							
7:	0000000000000000	99	FUNC	GLOBAL	DEFAULT	1	main
8:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	_ZSt3cin
9:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	_ZNSirsERi
10:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	_ZSt4cout
11:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	_ZNSolsEi
12:	0000000000000000	0	NOTYPE	GLOBAL	DEFAULT	UND	__stack_chk_fail

Посмотрим, где лежит `cin`:

```
readelf -s /lib/x86_64-linux-gnu/libstdc++.so.6 | grep St3cin
```

```
2306: 00000000002896a0 280 OBJECT GLOBAL DEFAULT 31
      _ZSt3cin@GLIBCXX_3.4
```

А вот так можно посмотреть количество символов в `libstdc++`:

```
readelf -s /lib/x86_64-linux-gnu/libstdc++.so.6 | wc -l
```

Мы можем переименовать `libstdc++` (главное не забыть вернуть всё как было) и увидеть, что наш “самодостаточный” бинарник не запустится. Всё потому, что он зависит от динамических библиотек.

С помощью `readelf -a a.out` можно посмотреть всю информацию о файле. А переменной окружения `LD_PRELOAD` можно указать другую версию библиотеки:

```
LD_PRELOAD=/lib/x86_64-linux-gnu/libstdc++.so.6.0.33 ./a.out
```

Если включить статическую линковку (это делается флагом `-static`), заголовок будет выглядеть иначе:

```
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 03 00 00 00 00 00 00 00 00
  Class:                               ELF64
  Data:                                   2's complement, little endian
  Version:                             1 (current)
  OS/ABI:                               UNIX - GNU
  ABI Version:                         0
  Type:                                 EXEC (Executable file)
  Machine:                             Advanced Micro Devices X86-64
  Version:                             0x1
  Entry point address:                  0x405e20
  Start of program headers:             64 (bytes into file)
  Start of section headers:            2415752 (bytes into file)
  Flags:                                0x0
  Size of this header:                  64 (bytes)
  Size of program headers:              56 (bytes)
  Number of program headers:            11
  Size of section headers:              64 (bytes)
  Number of section headers:            29
  Section header string table index:    28
```